OOP/OOPM Interview Question Bank

C++ + Java + Python + JavaScript

Level 1 — OOP Fundamentals

1. What is Object-Oriented Programming?
2. Why was OOP introduced?
3. What problems does OOP solve?
4. What are the main characteristics of OOP?
5. What are the four pillars of OOP?
6. What is a class?
7. What is an object?
8. What is the difference between a class and an object?
9. Is a class an object?
10. What is an instance?
11. What is object state?
12. What is object behavior?
13. What is object identity?
14. What is the difference between state, behavior, and identity?
15. What is an attribute/property/field?
16. What is a method?
17. What is the difference between a function and a method?
18. What happens when an object is created?
19. What happens when an object is destroyed?
20. How are objects stored in memory?
21. What is object-oriented modeling?
22. What is OOPM?
23. What are the advantages of OOP?
24. What are the disadvantages of OOP?
25. When should OOP be preferred?
26. Can a program be completely object-oriented?
27. What is a real-world example of OOP?
28. Why is OOP useful for large applications?
29. What is message passing between objects?
30. What is object communication?

Level 2 — Class and Object

31. How do you define a class?
32. How do you create an object?
33. How are class members accessed?
34. What is an instance variable?
35. What is a class/static variable?
36. What is an instance method?
37. What is a class/static method?
38. What is a constructor?
39. Why do we need constructors?

40. Can a class exist without explicitly defining a constructor?
41. Can an object exist without calling a constructor?
42. Can one class have multiple objects?
43. Can multiple objects share the same method?
44. What is the difference between class-level and object-level data?
45. What is object initialization?
46. What is object assignment?
47. What is object copying?
48. What is object reference?
49. What is object aliasing?
50. What happens when two references point to the same object?

Level 3 — Encapsulation

51. What is encapsulation?
52. Why is encapsulation required?
53. How does encapsulation protect data?
54. What is data hiding?
55. What is the difference between encapsulation and data hiding?
56. What are access modifiers?
57. What is public access?
58. What is private access?
59. What is protected access?
60. What is package/default/internal access?
61. Can private members be accessed outside a class?
62. How can private data be accessed safely?
63. What are getters and setters?
64. Why shouldn't fields always be directly exposed?
65. What is controlled access?
66. Can a getter modify object state?
67. Can a setter contain validation logic?
68. What is immutable data?
69. How does encapsulation help maintain invariants?
70. What is the difference between an encapsulated and non-encapsulated class?

Level 4 — Constructors & Object Lifecycle

71. What is a default constructor?
72. What is a parameterized constructor?
73. What is a copy constructor?
74. What is a constructor overloading?
75. Can constructors be inherited?
76. Can constructors be overridden?
77. Can constructors be private?
78. Why would you make a constructor private?
79. Can a constructor be static?
80. Can a constructor be abstract?

81. Can a constructor return a value?
82. What happens if a constructor throws an exception?
83. What is constructor chaining?
84. What is constructor delegation?
85. What is the order of constructor execution in inheritance?
86. What happens when a child object is created?
87. What is object destruction?
88. What is a destructor?
89. Which languages provide deterministic destruction?
90. What is garbage collection?
91. What is finalization?
92. Why is garbage collection different from destruction?
93. What is RAII?
94. Which language uses RAII?
95. What is object lifetime?

Level 5 — Inheritance

96. What is inheritance?
97. Why is inheritance used?
98. What is a parent/base/superclass?
99. What is a child/derived/subclass?
100. What is an "is-a" relationship?
101. What is single inheritance?
102. What is multilevel inheritance?
103. What is hierarchical inheritance?
104. What is multiple inheritance?
105. What is hybrid inheritance?
106. Which languages support multiple inheritance?
107. Which languages do not support class-based multiple inheritance?
108. Why doesn't Java support multiple class inheritance?
109. How does Python support multiple inheritance?
110. How can multiple inheritance be represented in Java?
111. What problems can inheritance create?
112. What is tight coupling in inheritance?
113. What is the inheritance hierarchy?
114. What is an indirect parent?
115. What members are inherited?
116. Are constructors inherited?
117. Are private members inherited?
118. Can a child access private members of its parent?
119. What is the difference between inheritance and composition?
120. When should inheritance be avoided?

Level 6 — super, Parent Initialization & Inheritance Behavior

121. What is super?

122. Why is super used?
123. How do you call a parent constructor?
124. How do you call a parent method?
125. Can you access a parent field using super?
126. What happens if the parent constructor is not explicitly called?
127. What is constructor chaining in inheritance?
128. What is the constructor execution order?
129. Can a child constructor call multiple parent constructors?
130. What happens when a parent constructor requires arguments?
131. Can a parent reference point to a child object?
132. Can a child reference point to a parent object?
133. What is upcasting?
134. What is downcasting?
135. What are the risks of downcasting?
136. What is instanceof/type checking?
137. How does inheritance affect method lookup?

Level 7 — Polymorphism

138. What is polymorphism?
139. Why is polymorphism important?
140. What are the types of polymorphism?
141. What is compile-time polymorphism?
142. What is runtime polymorphism?
143. What is static polymorphism?
144. What is dynamic polymorphism?
145. What is method overloading?
146. What is method overriding?
147. Difference between overloading and overriding?
148. Can constructors be overloaded?
149. Can constructors be overridden?
150. Can static methods be overloaded?
151. Can static methods be overridden?
152. Can private methods be overridden?
153. What is dynamic method dispatch?
154. What is late binding?
155. What is early binding?
156. What is virtual dispatch?
157. What is method resolution?
158. How does runtime decide which method to call?
159. Can polymorphism exist without inheritance?
160. How does JavaScript achieve polymorphism?
161. How does Python achieve polymorphism?
162. How does C++ implement runtime polymorphism?
163. How does Java implement runtime polymorphism?

Level 8 — Abstraction

164. What is abstraction?
165. Why is abstraction required?
166. What is the difference between abstraction and encapsulation?
167. What is an abstract class?
168. What is an abstract method?
169. Can an abstract class have constructors?
170. Can an abstract class have normal methods?
171. Can an abstract class have variables?
172. Can an abstract class have static methods?
173. Can an abstract class be instantiated?
174. Why can't an abstract class normally be instantiated?
175. What is an interface?
176. What is the difference between an abstract class and interface?
177. When should you use an abstract class?
178. When should you use an interface?
179. Can an interface contain implementation?
180. Can an abstract class implement an interface?
181. Can a class implement multiple interfaces?
182. Can an abstract class inherit another abstract class?
183. What happens if a subclass doesn't implement all abstract methods?
184. Can an abstract method be private?
185. Can an abstract method be static?

Level 9 — C++ Specific OOP Questions

Constructors / Destructors

186. What is a constructor in C++?
187. What is a destructor in C++?
188. Why does C++ have destructors?
189. Can a destructor be overloaded?
190. Can a destructor be virtual?
191. Why would a destructor be virtual?
192. What is constructor delegation?
193. What is an initializer list?
194. Why is an initializer list important?
195. What is the difference between initialization and assignment?
196. What is a copy constructor?
197. What is a copy assignment operator?
198. What is the difference between copy constructor and assignment operator?
199. What is the Rule of Three?
200. What is the Rule of Five?
201. What is the Rule of Zero?

Inheritance

202. What is public inheritance?
203. What is protected inheritance?

204. What is private inheritance?
205. What is the difference between inheritance modes?
206. What is multiple inheritance in C++?
207. What is the diamond problem?
208. How does C++ solve the diamond problem?
209. What is virtual inheritance?
210. What is an ambiguous base class?
211. What is object slicing?
212. How can object slicing occur?
213. How can object slicing be prevented?

Polymorphism

214. What is a virtual function?
215. Why do we need virtual functions?
216. What is a pure virtual function?
217. What is a virtual class?
218. What is an abstract class in C++?
219. What is a vtable?
220. What is a vptr?
221. How does virtual function dispatch work?
222. What happens when a virtual function is called through a base pointer?
223. What is function overriding in C++?
224. What is function overloading in C++?
225. What is function hiding?
226. What is the difference between overriding and hiding?
227. What does the override keyword do?
228. What does the final keyword do?
229. Can constructors be virtual?
230. Why can't constructors be virtual?
231. Can destructors be virtual?
232. Can static functions be virtual?
233. Can friend functions be virtual?
234. Can a pure virtual function have an implementation?

Friend Functions

235. What is a friend function?
236. Why are friend functions used?
237. Is a friend function a class member?
238. Can a friend function access private members?
239. Can a class have multiple friend functions?
240. Can one class be a friend of another?
241. What are the advantages of friend functions?
242. What are the disadvantages of friend functions?
243. Can friendship be inherited?
244. Is friendship mutual?
245. Is friendship transitive?
246. Can a friend function be overloaded?

Other C++ Concepts

247. What is operator overloading?
248. Which operators cannot be overloaded?
249. What is a member operator function?
250. What is a non-member operator function?
251. What is the difference between this pointer and normal pointer?
252. What is a const member function?
253. What is a mutable member?
254. What is a static data member?
255. What is a static member function?
256. Can a static member function access non-static members?
257. What is an inline member function?
258. What is a nested class?
259. What is an anonymous object?
260. What is RAII?
261. What is smart-pointer-based ownership?
262. How does unique_ptr relate to OOP design?
263. How does shared_ptr relate to object ownership?

Level 10 — Java Specific OOP Questions

264. What is a class in Java?
265. What is an object in Java?
266. What is the difference between primitive and reference types?
267. What is the this keyword?
268. What is super?
269. What is a static member?
270. What is a static method?
271. Can a static method access instance members?
272. Can a static method be overridden?
273. What is method overloading in Java?
274. What is method overriding in Java?
275. What are the rules for method overriding?
276. Can an overridden method reduce visibility?
277. What is covariant return type?
278. What is dynamic method dispatch?
279. What is an abstract class?
280. What is an interface?
281. What is the difference between abstract class and interface?
282. Can an interface have fields?
283. Can an interface have default methods?
284. Can an interface have static methods?
285. Can an interface have private methods?
286. Can a class implement multiple interfaces?
287. Can an interface extend another interface?
288. Can an interface extend multiple interfaces?

289. What is a functional interface?
290. What is a marker interface?
291. What is the final keyword?
292. Can a final class be inherited?
293. Can a final method be overridden?
294. Can a final variable be modified?
295. What is the difference between final, finally, and finalize?
296. What is an immutable class?
297. How do you create an immutable class?
298. Why is String immutable?
299. What is the Object class?
300. Which methods are inherited from Object?
301. What is equals()?
302. What is hashCode()?
303. Why should equals() and hashCode() be consistent?
304. What is toString()?
305. What is object cloning?
306. What is Cloneable?
307. What is shallow copy?
308. What is deep copy?
309. What is garbage collection in Java?
310. Can you force garbage collection?
311. What is an anonymous class?
312. What is a nested class?
313. What is an inner class?
314. What is a static nested class?
315. What is an enum?
316. Can an enum contain methods and fields?
317. What is composition in Java?
318. What is aggregation in Java?
319. What is association in Java?

Level 11 — Python Specific OOP Questions

320. What is a class in Python?
321. What is an object in Python?
322. What is self?
323. Why is self explicitly written?
324. Is self a keyword?
325. What is __init__()?
326. What is __new__()?
327. Difference between __new__() and __init__()?
328. What is __del__()?
329. Is __del__() equivalent to a C++ destructor?
330. What are instance variables?
331. What are class variables?

332. What is an instance method?
333. What is a class method?
334. What is a static method?
335. Difference between instance, class, and static methods?
336. What is @classmethod?
337. What is @staticmethod?
338. What is @property?
339. What is a getter in Python?
340. What is a setter in Python?
341. What is a deleter?
342. What is name mangling?
343. Does Python have truly private variables?
344. What does __variable mean?
345. What does _variable mean?
346. What is duck typing?
347. What is Python's approach to polymorphism?
348. What is method overriding in Python?
349. Does Python support method overloading?
350. How can Python simulate method overloading?
351. What is multiple inheritance?
352. What is MRO?
353. What is Method Resolution Order?
354. What is C3 linearization?
355. What is super()?
356. How does super() work internally?
357. What is an abstract base class?
358. What is the abc module?
359. What is @abstractmethod?
360. Can an abstract class have concrete methods?
361. Can Python instantiate an abstract class?
362. What are magic/dunder methods?
363. What is __str__()?
364. What is __repr__()?
365. Difference between __str__() and __repr__()?
366. What is __eq__()?
367. What is __lt__()?
368. What is operator overloading in Python?
369. What is __add__()?
370. What is __len__()?
371. What is __getitem__()?
372. What is __call__()?
373. Can objects be called like functions?
374. What are descriptors?
375. How does @property work internally?
376. What is __get__()?

377. What is __set__()?
378. What is __delete__()?
379. What is metaclass?
380. What is type in Python?
381. What is the relationship between class and metaclass?
382. Can a class itself be an object in Python?
383. What is monkey patching?
384. What is composition in Python?
385. What is aggregation in Python?
386. What is a protocol in Python?
387. What is structural subtyping?
388. What is typing.Protocol?

Level 12 — JavaScript OOP Questions

389. Is JavaScript an object-oriented language?
390. What is an object in JavaScript?
391. What is a prototype?
392. What is the prototype chain?
393. What is prototypal inheritance?
394. How is prototypal inheritance different from classical inheritance?
395. What is prototype?
396. What is __proto__?
397. Difference between prototype and __proto__?
398. What is Object.create()?
399. What is a constructor function?
400. What does new do internally?
401. What is a class in JavaScript?
402. Are JavaScript classes truly class-based internally?
403. What is the constructor method?
404. What is this in JavaScript?
405. How is this determined?
406. What is method binding?
407. What is bind()?
408. Difference between call(), apply(), and bind()?
409. What are public class fields?
410. What are private class fields?
411. What does #variable mean?
412. What are getters and setters?
413. What is inheritance using extends?
414. What does super() do?
415. What happens if super() isn't called in a derived constructor?
416. What is method overriding in JavaScript?
417. Does JavaScript support method overloading?
418. How can method overloading be simulated?
419. What is polymorphism in JavaScript?

420. What is encapsulation in JavaScript?
421. What is abstraction in JavaScript?
422. What are static methods?
423. What are static fields?
424. Can static methods access instance properties?
425. What is a static initialization block?
426. What are mixins?
427. What is composition over inheritance?
428. What is object destructuring in relation to objects?
429. What is Object.freeze()?
430. What is Object.seal()?
431. Difference between freeze() and seal()?
432. What is shallow copy?
433. What is deep copy?
434. What is structuredClone()?
435. What is instanceof?
436. How does instanceof work?
437. What is Object.is()?
438. Difference between ==, ===, and Object.is()?
439. What is a factory function?
440. Factory function vs constructor function?
441. What are closures in relation to encapsulation?
442. What is a Symbol?
443. Can Symbols be used for pseudo-private properties?
444. What is delegation in JavaScript?

Level 13 — Association, Aggregation & Composition

445. What is association?
446. What is aggregation?
447. What is composition?
448. Difference between association, aggregation, and composition?
449. What is a "has-a" relationship?
450. What is an "is-a" relationship?
451. What is a "uses-a" relationship?
452. Give a real-world example of association.
453. Give a real-world example of aggregation.
454. Give a real-world example of composition.
455. Can aggregation exist without the parent object?
456. Can composition exist without the parent object?
457. Why is composition often preferred over inheritance?
458. What is composition over inheritance?
459. What is delegation?
460. How does delegation differ from inheritance?

Level 14 — Advanced Polymorphism

461. What is subtype polymorphism?
462. What is parametric polymorphism?
463. What is ad-hoc polymorphism?
464. What is generic programming?
465. What is duck typing?
466. What is structural typing?
467. What is nominal typing?
468. What is static dispatch?
469. What is dynamic dispatch?
470. What is single dispatch?
471. What is multiple dispatch?
472. Which of these languages supports multiple dispatch directly?
473. What is method resolution?
474. What is overload resolution?
475. What is dynamic binding?
476. What is static binding?
477. What is covariant return type?
478. What is contravariance?
479. What is invariance?
480. How does polymorphism affect maintainability?

Level 15 — Memory & OOP

481. What is stack memory?
482. What is heap memory?
483. Where are objects generally allocated?
484. Where are local variables stored?
485. What is a reference?
486. What is a pointer?
487. Difference between pointer and reference?
488. What is object ownership?
489. What is object lifetime?
490. What is memory leak?
491. What is dangling reference?
492. What is dangling pointer?
493. What is shallow copy?
494. What is deep copy?
495. What is aliasing?
496. What is garbage collection?
497. What is reference counting?
498. What is tracing garbage collection?
499. What is automatic memory management?
500. How does object destruction differ between C++, Java, Python, and JavaScript?

Level 16 — Advanced Object Copying

501. What is shallow copying?

502. What is deep copying?
503. Why can shallow copying be dangerous?
504. What is copy semantics?
505. What is move semantics?
506. What is move constructor?
507. What is move assignment?
508. What is an rvalue?
509. What is an lvalue?
510. What is std::move()?
511. Why was move semantics introduced?
512. How does Python handle object assignment?
513. How does Python's copy.copy() work?
514. How does Python's copy.deepcopy() work?
515. How do Java references behave during copying?
516. How do you deep-copy an object in Java?
517. How do JavaScript object spread and Object.assign() perform copying?
518. What is structured cloning?

Level 17 — SOLID Principles

519. What are SOLID principles?
520. What is Single Responsibility Principle?
521. What is Open/Closed Principle?
522. What is Liskov Substitution Principle?
523. What is Interface Segregation Principle?
524. What is Dependency Inversion Principle?
525. Why are SOLID principles important?
526. What is a violation of SRP?
527. What is a violation of OCP?
528. What is a violation of LSP?
529. What is a violation of ISP?
530. What is a violation of DIP?
531. How does composition help SOLID design?
532. How does dependency injection relate to SOLID?
533. What is dependency injection?
534. What is constructor injection?
535. What is setter injection?
536. What is interface injection?
537. What is dependency inversion vs dependency injection?

Level 18 — Design & Architecture Questions

538. What is loose coupling?
539. What is tight coupling?
540. How do you reduce coupling?
541. What is high cohesion?
542. What is low cohesion?

543. Why should classes have high cohesion?
544. What is separation of concerns?
545. What is dependency?
546. What is dependency inversion?
547. What is delegation?
548. What is composition?
549. What is an abstraction layer?
550. What is an API boundary?
551. What is an interface contract?
552. What is programming to an interface?
553. What is programming to an implementation?
554. Why should implementation details be hidden?
555. What is extensibility?
556. What is maintainability?
557. What makes an OOP design scalable?
558. What makes an OOP design difficult to maintain?

Level 19 — Design Patterns

559. What is a design pattern?
560. Why are design patterns used?
561. What is the Singleton pattern?
562. What are the problems with Singleton?
563. How can Singleton be implemented in C++?
564. How can Singleton be implemented in Java?
565. How can Singleton be implemented in Python?
566. How can Singleton be implemented in JavaScript?
567. What is Factory pattern?
568. Factory vs constructor?
569. What is Abstract Factory?
570. What is Builder pattern?
571. What is Prototype pattern?
572. What is Adapter pattern?
573. What is Decorator pattern?
574. What is Facade pattern?
575. What is Proxy pattern?
576. What is Observer pattern?
577. What is Strategy pattern?
578. What is Command pattern?
579. What is Template Method pattern?
580. What is Dependency Injection pattern?
581. Which design patterns are commonly used in real applications?

Level 20 — Language Comparison Questions

582. How is OOP implemented in C++?
583. How is OOP implemented in Java?

584. How is OOP implemented in Python?
585. How is OOP implemented in JavaScript?
586. Which of these languages supports multiple inheritance?
587. Which supports method overloading directly?
588. Which supports method overriding?
589. Which supports operator overloading?
590. Which supports friend functions?
591. Which supports virtual functions?
592. Which supports interfaces?
593. Which supports abstract classes?
594. Which uses garbage collection?
595. Which uses deterministic destruction?
596. Which uses prototypes?
597. Which uses MRO?
598. What is the equivalent of this in each language?
599. What is the equivalent of super in each language?
600. How is encapsulation implemented in each language?
601. How is abstraction implemented in each language?
602. How is polymorphism implemented in each language?
603. How does inheritance differ among the four?
604. How does memory management differ among the four?
605. How does object creation differ among the four?

Level 21 — Tricky Interview Questions

606. Can you achieve polymorphism without inheritance?
607. Can you achieve abstraction without an abstract class?
608. Can you achieve encapsulation without private variables?
609. Is inheritance always an example of polymorphism?
610. Is every inheritance relationship polymorphic?
611. Is every object an instance of a class?
612. Is a class an object?
613. Can an object exist without a class?
614. Can a class exist without objects?
615. Can you override a private method?
616. Can you override a static method?
617. Can you overload a static method?
618. Can you overload a constructor?
619. Can you inherit a constructor?
620. Can an abstract class have a constructor?
621. Can an abstract class have concrete methods?
622. Can an interface contain implementation?
623. Can an interface be instantiated?
624. Can a final method be overridden?
625. Can a final class be inherited?
626. Can a virtual function be static?

627. Can a constructor be virtual?
628. Can a destructor be virtual?
629. Can a friend function be inherited?
630. Can friendship be inherited?
631. Can private data be inherited?
632. Can a child directly access parent private data?
633. Is composition always better than inheritance?
634. Is multiple inheritance always bad?
635. Is abstraction the same as hiding implementation?
636. Is encapsulation the same as making everything private?
637. Is polymorphism the same as method overriding?
638. Is method overloading runtime polymorphism?
639. Is duck typing polymorphism?
640. Can polymorphism exist without method overriding?

Level 22 — Practical/Scenario-Based OOP Questions

641. Design a Bank Account class.
642. Design an ATM using OOP.
643. Design a Library Management System.
644. Design a Parking Lot.
645. Design an Employee Management System.
646. Design an Online Shopping System.
647. Design a Food Delivery System.
648. Design a Hospital Management System.
649. Design a Ride-Sharing System.
650. Design a Movie Ticket Booking System.
651. Design a Chess game using OOP.
652. Design a Tic-Tac-Toe game using OOP.
653. Design a Car Rental System.
654. Design a Hotel Booking System.
655. Design a Quiz application using OOP.
656. Identify classes and objects in a given real-world problem.
657. Identify relationships between classes.
658. Decide whether a relationship should use inheritance or composition.
659. Identify where abstraction should be introduced.
660. Identify where polymorphism should be used.
661. Identify violations of encapsulation.
662. Identify tight coupling in a given design.
663. Refactor a badly designed class.
664. Apply SOLID principles to an existing design.
665. Design a system that supports adding new features without modifying existing classes.

Level 23 — Code-Tracing Interview Questions

666. What will this constructor code print?
667. In what order will constructors execute?

668. In what order will destructors execute?
669. Which overloaded method will be selected?
670. Which overridden method will execute?
671. What happens when a parent reference points to a child object?
672. What happens when a child object is assigned to a parent reference?
673. Which method is called through a virtual function?
674. What happens without virtual in C++?
675. What happens when a static method is called through an object?
676. What happens when a private member is accessed indirectly?
677. What happens when a Python property setter is called?
678. What happens when Python uses multiple inheritance?
679. What method does Python find first according to MRO?
680. What does JavaScript's prototype chain search?
681. What does new do internally in JavaScript?
682. What does super() do internally?
683. What happens when this changes?
684. What happens when an object is shallow-copied?
685. What happens when two references point to the same object?

Level 24 — Expert-Level OOP

686. What is structural vs nominal subtyping?
687. What is behavioral subtyping?
688. What is the Liskov Substitution Principle formally describing?
689. What is double dispatch?
690. What is multiple dispatch?
691. What is type erasure?
692. What are generics?
693. What are templates?
694. Java generics vs C++ templates?
695. What is reflection?
696. How does reflection affect OOP design?
697. What are annotations?
698. What are decorators in Python?
699. What are decorators in JavaScript?
700. What is metaprogramming?
701. What is a metaclass?
702. What is runtime type information?
703. What is RTTI in C++?
704. What is dynamic_cast?
705. What is typeid?
706. What is introspection?
707. What is dependency inversion at architectural scale?
708. What is object capability?
709. What is immutability?
710. Why is immutability useful in concurrent applications?

711. What is thread-safe object design?
712. What makes an object thread-safe?
713. What is reentrant behavior?
714. What is anemic domain model?
715. What is a rich domain model?
716. What is domain-driven design?
717. What is bounded context?
718. What is aggregate in Domain-Driven Design?
719. What is entity vs value object?
720. What is identity equality vs value equality?